Retrace: Product Requirements Document

Hindsight, without the bias.

Version 0.6 | 18 October 2026 | Draft for review

1. Overview

Retrace is a multi-tenant web platform that turns the scattered record of an incident into a finished, blameless postmortem. A team connects Slack, GitHub, and PagerDuty once. When an incident ends, someone tells Retrace what happened in a sentence or two, such as the website went down or we suspect a cyberattack. Retrace finds the evidence on its own, builds the timeline, drafts the postmortem, checks the language for blame, and tracks the follow-up work until it is done.

This document defines what Retrace is, who it is for, and what v1 must do. The system design and database design documents build on it.

2. Problem and opportunity

After an incident, someone has to write the postmortem, and the process is slow and biased:

- The evidence lives in Slack threads, alert pages, pull requests, and deployment logs. People hunt through each one by hand.
- Industry sources report 60 to 90 minutes of manual reconstruction per incident for a simple one, and several hours for a severe one.
- Drafts written from memory slip into hindsight bias and finger-pointing, which damages the blameless culture teams want.
- Action items are written down and then forgotten. Industry benchmarks put the repeat-incident rate at around 21 percent.
- Compliance reviews such as SOC 2 ask for consistent, auditable incident records that most teams assemble by hand.

These figures come from vendor and community research, so they are directional, not guaranteed. Retrace should measure its own results (see section 14).

3. Product principles

1. **Zero setup per incident.** The author describes what happened. The AI finds the evidence.
2. **Evidence first.** Every claim in a draft links to the timeline events behind it. Unsupported claims are labeled as hypotheses.
3. **Blameless by default.** The product flags and rewrites language that blames individuals.
4. **Humans decide.** Nothing is published, assigned, or sent externally without a person approving it.
5. **Private by design.** Each company's data is isolated, and the AI reads only the sources an admin has allowed.
6. **Model-flexible.** Every organization sets its AI model, and a default model works out of the box.

4. Goals and non-goals

Goals

1. Cut the time from incident end to first draft from hours to minutes.
2. Produce a traceable timeline where every event links back to its source.
3. Keep postmortems blameless and consistent across teams.
4. Make sure action items get done, and surface repeat incidents.
5. Keep each organization's data strictly separate from every other organization's.
6. Provide exports and audit-ready reports teams can hand to leadership and auditors.

Non-goals for v1

- Replacing incident response, paging, or on-call tools.
- Publishing or notifying customers automatically without human review.
- Integrations beyond Slack, GitHub, and PagerDuty.
- A hosted self-service billing system.
- Native mobile apps.
- Open-sourcing the project or supporting self-hosted installs (both may come later).

5. Users and personas

Persona	Who they are	What they need
Incident author	On-call engineer or SRE who owns the postmortem	A fast first draft they only need to review and correct
Reviewer	Teammate or engineer involved in the incident	Clear evidence behind each claim and easy comments
Manager	Optional role: engineering lead who approves postmortems	A trustworthy draft, visible action items, and an approval step
Org admin	Platform owner for the company workspace	Control over members, roles, integrations, allowed sources, and AI model settings
Viewer	Anyone invited who needs read access	Finished postmortems and trends, without edit rights
Auditor or leadership reader	Compliance or leadership consumer of reports	Audit-ready exports with approval history and evidence

6. The seven core features

#	Core feature	What it means for the user	Detailed in
1	Automatic incident timeline	One ordered timeline built from chat, code, and alert tools, with sources linked	8.6
2	AI-generated first draft	A complete postmortem draft grounded in the timeline	8.7
3	Blameless language check	Finger-pointing phrases are flagged and rewritten in neutral terms	8.8
4	Review workflow	Editing, comments, versions, and manager approval	8.9

#	Core feature	What it means for the user	Detailed in
5	Action item tracking	Owners, due dates, and reminders for stalled tasks	8.11
6	Private workspaces with role-based access	Separate, invite-only workspace per company	8.1, 8.2
7	Analytics and audit-ready reports	Repeat-incident and recovery-time analytics, plus exportable reports	8.10, 8.12

7. End-to-end user journey

One-time setup (org admin)

1. Create the company workspace and invite members by email.
2. Connect Slack, GitHub, and PagerDuty.
3. Choose which Slack channels and GitHub repositories the AI is allowed to read (an allowlist, with sensible defaults such as public incident channels).
4. Add services to the service catalog and link each one to its repositories, Slack channels, and PagerDuty service. This is done once and can be edited at any time.
5. Set the AI model (required): keep the Retrace default model, or enter another model with its provider endpoint and API key.

Per incident (author)

1. An incident is resolved in PagerDuty and appears in Retrace, or the author creates one manually.
2. The author writes a short description of what happened and picks the severity if it is not already set. That is the only required input.
3. Retrace discovers the evidence automatically: it uses the incident's service, time window, linked channels, repositories, and deployments, and it searches allowed sources for related activity.
4. The author sees a short evidence preview showing what was found and can remove or add items. This step is optional and takes seconds.
5. Retrace builds the timeline and drafts the postmortem.
6. The blameless check runs on the draft and suggests rewrites.

7. The author edits, reviewers comment, and the author submits for review.
8. A manager approves, and the postmortem is published to the org.
9. Action items are tracked, and reminders go out for items that stall.

8. Functional requirements

Priority key: **P0** must have for v1, **P1** should have for v1 if time allows, **P2** later. Each requirement has an ID for traceability.

8.1 Accounts and sign-in

- **AUTH-1 (P0)** Sign up and sign in with email and password.
- **AUTH-2 (P0)** Sign in with OAuth using GitHub and Google.
- **AUTH-3 (P0)** Email verification and password reset.
- **AUTH-4 (P0)** Sessions expire, and users can sign out of all devices.
- **AUTH-5 (P1)** Optional two-factor authentication.
- **AUTH-6 (P1)** Rate limiting and lockout on repeated failed sign-ins.

8.2 Organizations, invitations, and roles

- **ORG-1 (P0)** Every record belongs to exactly one organization. A user can belong to several organizations and switches between them.
- **ORG-2 (P0)** Membership is invite-only. There is no public sign-up into an existing organization. Admins invite by email, and invitations expire and can be revoked.
- **ORG-3 (P0)** Admin and Member are the required roles and exist in every organization. Manager and Viewer are optional roles that an Admin can enable. In the first release only Admin and Member are built, and Admins approve postmortems until the optional roles arrive. Permissions follow the matrix below.
- **ORG-4 (P0)** Admins can change roles and remove members. Removing a member keeps their authored content but ends their access.
- **ORG-5 (P0)** The first user who creates an organization becomes its Admin. An organization must always have at least one Admin.
- **ORG-6 (P1)** Teams within an organization, so incidents and postmortems can be grouped by group of services.
- **ORG-7 (P1)** Organization deletion that removes all of its data.

Action	Admin (required)	Manager (optional)	Member (required)	Viewer (optional)
Invite or remove members, change roles, enable optional roles	Yes	No	No	No
Connect integrations, set allowed sources, set AI model	Yes	No	No	No
Manage service catalog	Yes	Yes	No	No
Create incidents and generate drafts	Yes	Yes	Yes	No
Edit and comment on postmortems	Yes	Yes	Yes	Comment only
Approve postmortems	Yes	Yes	No	No
View published postmortems and analytics	Yes	Yes	Yes	Yes
Export reports	Yes	Yes	Yes	Yes
View audit log	Yes	No	No	No

8.3 Integrations and source permissions

- **INT-1 (P0)** Slack: connect a workspace with OAuth and read messages and threads from allowed channels within a time window.
- **INT-2 (P1, stretch goal in the first release)** GitHub: connect with an app installation. Read merged pull requests, commits, releases, deployments, and workflow runs for allowed repositories.
- **INT-3 (P0)** PagerDuty: read incidents, alerts, acknowledgements, escalations, notes, and resolution times, and create incidents in Retrace from resolved PagerDuty incidents (by webhook or periodic sync).
- **INT-4 (P0)** Admins define an allowlist of channels and repositories. The AI never reads outside it. Direct messages and private channels are excluded unless an admin explicitly adds a private channel.

- **INT-5 (P0)** Integration tokens are stored encrypted. Admins can disconnect any integration, which stops all further reads.
- **INT-6 (P0)** Evidence collection runs as a background job with retries, pagination, and respect for each provider's rate limits. Failures are shown clearly, never hidden.
- **INT-7 (P1)** Re-sync evidence for an incident on demand.
- **INT-8 (P1)** Optional redaction rules, such as removing email addresses or tokens, applied before evidence is stored or sent to a model.

8.4 Service catalog

- **SVC-1 (P0)** Admins and managers can create services with a name, owning team, and description.
- **SVC-2 (P0)** Each service can be linked to repositories, Slack channels, and a PagerDuty service.
- **SVC-3 (P0)** The catalog is the main hint for evidence discovery. If a service has no links, discovery falls back to PagerDuty data and keyword search over allowed sources.
- **SVC-4 (P1)** Suggest links automatically from names, so setup is mostly confirmation.

8.5 Incidents

- **INC-1 (P0)** Create an incident from PagerDuty or manually.
- **INC-2 (P0)** Fields: title, short description of what happened, incident type (outage, performance, security, data, other), severity, status, start time, end time, affected services, and responders.
- **INC-3 (P0)** The only required author input is the short description. Everything else is filled from PagerDuty or inferred, and can be corrected.
- **INC-4 (P0)** The incident detail page shows the timeline, the draft, review status, and action items in one place.
- **INC-5 (P1)** Duplicate detection when two PagerDuty incidents describe the same event, with a merge option.

8.6 Evidence discovery and automatic timeline (core feature 1)

- **EVD-1 (P0)** Starting from the description, severity, affected services, and time window, Retrace discovers relevant evidence without the author choosing sources.

- **EVD-2 (P0)** Discovery uses, in order: the PagerDuty incident record, the service catalog links, deployments and merges near the incident window, and a search over allowed sources for related messages and commits. The incident type shapes the search. For a security incident it also looks for access changes, secret rotations, and security alerts.
- **EVD-3 (P0)** The time window is derived automatically, from shortly before the first alert to shortly after resolution, with a visible default the author can change.
- **EVD-4 (P0)** All evidence is normalized into timeline events with source, original link, author, timestamp, event type, and a confidence value.
- **EVD-5 (P0)** The author sees an evidence preview before drafting and can remove irrelevant items or add missing ones, including by pasting a link.
- **EVD-6 (P0)** The author can edit, hide, or add events manually. Manual events are marked as such.
- **EVD-7 (P0)** Timeline events are ordered by time. Where sources disagree on time, the discrepancy is shown instead of hidden.
- **EVD-8 (P1)** Mark key moments: first symptom, detection, escalation, mitigation, resolution.
- **EVD-9 (P1)** Group related events into phases such as detection, diagnosis, mitigation, and recovery.

8.7 AI-generated first draft (core feature 2)

- **AI-1 (P0)** Generate a draft with these sections: summary, impact, timeline, root cause, contributing factors, detection, response, what went well, what went poorly, lessons learned, and action items.
- **AI-2 (P0)** Each factual claim cites one or more timeline events. Claims without evidence are marked as hypotheses or open questions.
- **AI-3 (P0)** The AI model is a required organization setting. New organizations start with the Retrace default model already selected, and admins can switch to another model by entering the provider, model name, endpoint, and API key. A connection test runs when the setting is saved. Retrace talks to models through an adapter so new providers can be added.
- **AI-4 (P0)** Generation runs as a background job with progress shown. A failed or partial generation can be retried without losing collected evidence.
- **AI-5 (P0)** Drafts are versioned. Regenerated text arrives as a suggestion that the author accepts or rejects, so it never overwrites human edits.

- **AI-6 (P0)** The root-cause section separates observed facts, evidence-backed hypotheses, and open questions. It never presents speculation as confirmed cause.
- **AI-7 (P1)** Regenerate one section with extra guidance from the author.
- **AI-8 (P1)** Show model name, token usage, and cost estimate per generation.
- **AI-9 (P2)** Use previous postmortems and runbooks as retrieval context for related incidents.

8.8 Blameless language check (core feature 3)

- **BL-1 (P0)** Scan the draft and any human edits for phrases that assign personal fault, judge individuals, or use counterfactual blame (for example, naming a person as the cause, or saying someone should have known).
- **BL-2 (P0)** Each flagged phrase shows a short reason and a suggested neutral rewrite that focuses on the system, process, or conditions.
- **BL-3 (P0)** The author can accept, edit, or dismiss each suggestion. Accepted changes are recorded in the version history.
- **BL-4 (P0)** The check runs automatically before a draft can be submitted for review, and the result is visible to the reviewer.
- **BL-5 (P1)** Option to display responders by role (for example, on-call engineer) instead of by name in the published version.

8.9 Review workflow and approval (core feature 4)

- **REV-1 (P0)** Rich text editor for the postmortem (see 8.9a), with autosave and no manual save step.
- **REV-2 (P0)** Postmortem states: Draft, In review, Changes requested, Approved, Published.
- **REV-3 (P0)** Comments on sections or selected text, with replies and resolve.
- **REV-4 (P0)** Named version snapshots showing who changed what and when, taken automatically on AI generation, submit, approval, and publish, and on demand, with the ability to compare and restore versions.
- **REV-5 (P0)** Manager approval is an explicit step. Only Admins and Managers can approve. If the optional Manager role is not enabled, Admins approve. The approver can approve or request changes with a note.
- **REV-6 (P0)** An author cannot approve their own postmortem.

- **REV-7 (P0)** Approval history is recorded with approver, time, and the version approved. Approved and published postmortems are read-only. A Reopen action stores a snapshot and returns the postmortem to In review.
- **REV-8 (P1)** Assign specific reviewers and notify them in the app and by email.
- **REV-9 (P1)** Review checklist the approver completes (impact stated, evidence cited, blameless check passed, action items assigned).

8.9a Editor experience

The postmortem editor should feel as smooth as a modern collaborative writing workspace, with AI built into the text.

- **ED-1 (P0)** Typing feels instant. Edits appear on the screen first and save in the background, with a visible saved status.
- **ED-2 (P0)** Inline AI actions: select text and choose an action (make blameless, clearer, shorter, expand using evidence) or type a custom instruction. The result streams in as a suggestion shown as a change, and the author accepts or rejects it.
- **ED-3 (P0)** The AI sees the whole postmortem and the timeline for these actions, and cites timeline events for any new fact it adds.
- **ED-4 (P0)** Citation chips in the text open the linked timeline event in a side panel.
- **ED-5 (P0)** Inline comment threads anchored to selected text that stay attached as the text moves, with replies and resolve (extends REV-3).
- **ED-6 (P0)** Role rules apply live: Admins and Members can edit, and nobody edits while the postmortem is Approved or Published until it is reopened.
- **ED-7 (P0)** One author at a time in the first release. If the same postmortem is open in two places, the second save is detected and the person is asked to reload, so work is never silently overwritten.
- **ED-8 (P1)** Slash commands and keyboard shortcuts for common blocks and actions.
- **ED-9 (P1)** Suggestion mode: a collaborator proposes tracked changes that the author accepts or rejects.
- **ED-10 (Later)** Real-time co-editing with live cursors, names, and colors. Changes appear for others within about one second, with at least 10 simultaneous editors. The first editor is built so this can be added without a rewrite.
- **ED-11 (Later)** Offline buffer so a longer disconnect does not lose work.
- **ED-12 (Later)** Voice dictation for small edits.

8.10 Export and audit-ready reports (core feature 7)

- **EXP-1 (P0)** Export any postmortem to PDF and to Markdown.
- **EXP-2 (P0)** Exports include the timeline with source references and the current version label.
- **EXP-3 (Later)** Audit-ready report: a PDF that bundles the approved postmortem, its evidence citations, the approval history, the action items with their status, and the relevant audit log entries for that incident.
- **EXP-4 (P1)** Bulk export of postmortems for a date range or service.
- **EXP-5 (P1)** Export an organization's full data as a machine-readable archive.
- **EXP-6 (P2)** Share a read-only link to a published postmortem, with expiry.

Retrace organizes evidence to support audits. It does not claim that an organization is compliant with any framework.

8.11 Action items and reminders (core feature 5)

- **ACT-1 (P0)** Create action items from the draft or by hand, with title, description, owner, due date, priority, success criterion, and status.
- **ACT-2 (P0)** AI-proposed action items are suggestions. Nothing is assigned until a person confirms the owner and due date.
- **ACT-3 (P0)** Statuses: Open, In progress, Blocked, Done, Dropped (with a reason).
- **ACT-4 (P0)** An action item is **stalled** if it is not Done and either it is past its due date, or it has had no update for 7 days (configurable per organization).
- **ACT-5 (P0)** Reminders: the owner is notified before the due date and when an item stalls. If it stays stalled for a further 3 days (configurable), the owner's Manager (or the Admins if the Manager role is not enabled) is also notified. Notifications go through email, and in the app.
- **ACT-6 (P0)** A dashboard of open, stalled, and overdue action items, filterable by owner, service, and priority.
- **ACT-7 (P1)** Create a linked GitHub issue from an action item, with status kept in sync.
- **ACT-8 (P1)** Slack reminders in addition to email.
- **ACT-9 (P1)** Verification step: when an item is marked Done, the owner adds a note or link showing the fix.

8.12 Analytics (core feature 7)

- **ANL-1 (P0)** Recovery time metrics: time to detect, time to acknowledge, time to mitigate, and time to resolve, per incident, and as averages and trends by service, severity, and period.
- **ANL-2 (P0)** Incident volume by service, severity, type, and period.
- **ANL-3 (Later)** Repeat-incident detection. An incident is flagged as a possible repeat of an earlier one when it affects the same service and its root cause matches an earlier one, using the root-cause category plus semantic similarity of the root-cause text above a configurable threshold, within a configurable lookback window (default 180 days). Authors and managers can confirm or dismiss each suggested link, and only confirmed links count in reports.
- **ANL-4 (Later)** Repeat-incident rate: confirmed repeats divided by total incidents in the period, shown as a trend and by service.
- **ANL-5 (P0)** Action item health: completion rate, average time to close, and stalled count.
- **ANL-6 (P1)** Top recurring causes and contributing factors, clustered across incidents.
- **ANL-7 (P1)** Postmortem timeliness: time from resolution to first draft, and from draft to approval.
- **ANL-8 (P2)** Link repeats to the action items that were supposed to prevent them.

8.13 Dashboard and search

- **DSH-1 (P0)** Incident list with filters by severity, status, service, type, and date.
- **DSH-2 (P0)** Home overview: recent incidents, postmortems awaiting review, stalled action items, and headline metrics.
- **DSH-3 (P1)** Full-text search across postmortems, timelines, and action items.
- **DSH-4 (P1)** Saved filters.

8.14 Audit log and administration

- **ADM-1 (P0)** Audit log records sign-ins, role changes, invitations, integration changes, source allowlist changes, AI model changes, approvals, publishes, exports, and deletions.
- **ADM-2 (P0)** Audit log entries cannot be edited or deleted by users, and each entry includes a hash of the previous entry so tampering is detectable.
- **ADM-3 (P1)** Configurable retention for evidence and drafts.

- **ADM-4 (P1)** Admin settings page for notification thresholds (stall days, reminder timing) and analytics thresholds.

9. Status models

Object	States	Notes
Incident	Open, Resolved, Postmortem in progress, Closed	Closed when the postmortem is published and no critical action items are open
Postmortem	Draft, In review, Changes requested, Approved, Published	Editing after approval returns it to In review
Action item	Open, In progress, Blocked, Done, Dropped	Stalled is a computed flag, not a state
Evidence item	Collected, Included, Excluded, Failed	Excluded items stay out of drafts but remain in history
Generation job	Queued, Running, Succeeded, Failed	Failed jobs can be retried

10. AI model configuration

- The AI model is a required organization setting. Every new organization starts with the Retrace default model already selected, so drafting works without any setup, and admins can change it at any time.
- The default model is a small, low-parameter model served through the Azure education resources available to the project. It is chosen to keep compute and cost low. The exact model is an open question.
- To use another model, an admin enters the provider, model name, API endpoint, and API key. The key is stored encrypted, and a connection test runs on save.
- All model calls go through one internal adapter, so changing providers does not change product behavior. Local or self-hosted models can be added later through the same adapter.

- The default model is small and shared, so each organization has a usage limit on it (for example, a daily cap on draft generations), and evidence is split into short pieces so the model is not overloaded.
- Prompts, output schemas, and citation rules are versioned in the codebase, and every draft records which model produced it.
- Evidence sent to a hosted model leaves the organization tenant. The settings page must say so plainly.

11. Non-functional requirements

Security and tenant isolation

- Each organization has its own database, and each service has its own database role per organization, so a bug in one service cannot read another service's data or another organization's data.
- Automated tests try to cross tenant and service boundaries and must fail to do so.
- Secrets and tokens are encrypted at rest with a separate key per organization, protected by a master key in a key vault. Passwords use a modern salted hash. All traffic uses HTTPS. Data stores have no public addresses, and only the frontend and the gateway are reachable from the internet.
- Role checks run on the server for every action, never only in the interface.
- Evidence stays inside the organization's tenant and is sent to a model only for the incident being drafted.

Privacy

- Collect only what the allowlist and time window cover.
- Direct messages and private channels are off by default.
- Deleting an incident deletes its evidence, drafts, and comments.

Performance

- Dashboard and list pages load in under 2 seconds for typical data sizes.
- Evidence collection and drafting run as background jobs with visible progress.
- A first draft is ready within 10 minutes of starting for a typical incident, and the service stays usable while jobs run.

Reliability and quality

- Integration and model failures are retried with backoff and shown clearly.
- A failed generation never loses collected evidence.
- Drafts are grounded. At least 90 percent of factual claims carry a citation in test incidents.

Usability

- The per-incident path needs one required input, the short description.
- Responsive layout that works on a laptop and a tablet. Accessible color contrast and keyboard navigation for core flows.

12. Proposed tech stack

This is the stack chosen for the project. It runs on Azure.

Layer	Choice	Reason
Frontend	React single-page app on Azure Static Web Apps, with Tailwind CSS and shadcn/ui	Isolated from the backend and fast to build
Gateway	Traefik container	Routing, TLS, token verification, and rate limiting in one place
Services	Microservices on Azure Container Apps: Core, Ingestion, AI, and Notifications behind the gateway, with a Realtime service later	Independent scaling and upgrades, and clear security boundaries
Backend frameworks	Django REST Framework in Core, Ingestion, and Notifications. Django Ninja in the AI service.	Full use of DRF, with async streaming and typed schemas where AI work needs them
Database	Azure Database for PostgreSQL: one central control database plus one database per organization, found through a tenant registry	Strong isolation, clean deletion, and independent growth

Layer	Choice	Reason
Search	PostgreSQL full-text search and trigram similarity, with pgvector later	No extra search service
Jobs and events	Celery with Redis for jobs, Redis Streams for events between services	Fewest moving parts, and the broker can be swapped later
AI layer	LLM API behind a model adapter, with a small default model on Azure and a required per-organization model setting	Model-flexible
Editor	TipTap (ProseMirror) storing JSON documents, with Yjs added later for real-time editing	Fits the 8-week plan with a path to real time
File storage	Azure Blob Storage	Exported PDFs and Markdown, and evidence snapshots
Secrets	Azure Key Vault, with a separate data key per organization	One organization's key cannot decrypt another's secrets
Email	Azure Communication Services Email or any SMTP provider	Invitations, password resets, and reminders. Not yet chosen.

The system design document describes how these pieces fit together.

13. Demo and data plan

Retrace is built as a real product with working integrations. For the capstone demo website, a public incident dataset from Kaggle will be loaded as seed data so the dashboard, analytics, and repeat-incident views have realistic volume.

- Real integrations stay fully functional and are demonstrated live with a test Slack workspace, a test GitHub repository, and a PagerDuty sandbox or developer account.
- Kaggle incident datasets are usually tabular records (for example, IT service tickets) and do not contain chat or code history. Seed data will therefore be paired with generated sample messages and deployments so timelines look complete.

- Seeded data is clearly marked as demo data in the database so it never mixes with real tenant data.
- The demo should show the full loop on at least one live incident: describe, discover evidence, draft, blameless check, review, approve, export, and track action items.

14. Success metrics

Metric	Target for the capstone	How to measure
Time from incident end to reviewable draft	Under 10 minutes	Timestamps in the app
Required author inputs per incident	One (short description)	Count of required fields
Draft claims with a cited source	90 percent or more	Sample review of test incidents
Relevant evidence recovered	90 percent of labeled key events found	Compare with a hand-labeled test incident
Cross-tenant data leaks in testing	Zero	Automated isolation tests
Blameless flags accepted or edited by authors	Tracked	Suggestion outcomes
Stalled action items surfaced before 14 days	100 percent	Reminder logs

A small evaluation package should be created: one or two labeled incidents with known events and known root cause, used to compare any chosen AI model.

15. Release scope

Phase	Weeks	Scope
1. Foundation	1 to 2	Authentication, organizations, invitations, Admin and Member roles, automatic tenant database creation, incidents (manual and from PagerDuty), audit log
2. Core loop	3 to 5	PagerDuty and Slack ingestion, service catalog, allowlists, evidence discovery and timeline, AI drafting with citations, blameless check, single-author editor with inline AI and comments, review and approval, PDF and Markdown export. Stretch: GitHub deployments and merged pull requests
3. Close the loop	6 to 8	Action items, stalled reminders and email, recovery-time analytics, Azure deployment, demo data, tests
Later	After the first release	Real-time co-editing, repeat-incident detection, audit-ready reports, optional Manager and Viewer roles, full GitHub integration, richer analytics, two-factor authentication, organization data export

The build is about 8 weeks and done by one person, so the first release is a thin slice that completes the core loop (describe, discover, draft, review, approve, track). If the schedule slips, cut in this order: email delivery, analytics, GitHub, then extra export formats. Tenant isolation tests and draft validation are not cut.

16. Risks and mitigations

Risk	Impact	Mitigation
Automatic discovery pulls irrelevant or sensitive material	Wrong draft or privacy issue	Admin allowlists, service catalog hints, evidence preview, redaction rules

Risk	Impact	Mitigation
AI invents facts	Loss of trust	Mandatory citations, hypothesis labeling, human approval
Results vary across AI models, and the small default model is weaker	Inconsistent quality	Versioned prompts, strict output schema, short evidence chunks, evaluation package, usage limits on the default model, model shown on each draft
Cross-tenant data exposure	Severe	Org-scoped queries and jobs, isolation tests
API rate limits and sandbox limits on Slack, GitHub, PagerDuty	Slow or partial ingestion	Background jobs, pagination, retries, test accounts
Kaggle data lacks chat and code context	Weak demo timelines	Generated sample messages and deployments paired with seed records
Scope too large for the semester	Incomplete demo	Phased release plan, P0 first
Sensitive chat content sent to hosted models	Privacy concern	Clear warning in settings, redaction rules, organization-chosen model

Added risk: real-time editing. It is the largest single build item, so it is scheduled for a later phase. The first release stores documents in a format that can be converted for real-time editing, so the later work does not require a rewrite.

17. Open questions

- Which Azure model will be the default, and what quota does the education subscription allow?
- Which email provider sends invitations and reminders: Azure Communication Services Email or another SMTP service?
- Which Kaggle dataset will be used for demo data, and which custom domain name will serve the app and api subdomains?

- Are the defaults acceptable: 7 days without an update marks an item stalled, and 3 more days escalates to the Admins (or the Manager once that role exists)?
- Should the project be open-sourced after the capstone is graded?

18. Glossary

- **Postmortem:** a written review of an incident covering what happened, why, and how to prevent it.
- **Blameless:** focusing on systems and processes, not individual fault.
- **Tenant:** one organization using Retrace, with its data kept separate.
- **Evidence:** raw items collected from integrations, such as messages, alerts, and commits.
- **Timeline event:** a normalized, time-stamped entry built from evidence.
- **Service catalog:** the org's list of services linked to repositories, channels, and PagerDuty services.
- **Allowlist:** the channels and repositories an admin has permitted the AI to read.
- **Stalled action item:** an unfinished item that is overdue or has had no update for the configured number of days.
- **Repeat incident:** an incident whose service and root cause match an earlier incident.